

Design of Hardware Accelerators

Indice

Introduction to SoCs and heterogeneous SoCs	1
Offloading computation	2
ASICs	2
FPGA	3
Design flows	3
Open challenges	3
Latency insensitive design	4
Communication synthesis	5
Communication protocols	6
Hardware security	6
HLS	6
Control and datapath extraction	7
High-level functionality description	7
Scheduling and binding	8
Scheduling	8
Binding	10
Microarchitecture creation	11
Memories and memory operations	11
Balancing multiple accelerators	13
Hardware security	14
Dynamic Information Flow Tracking	14
Obfuscation	14
Watermarking	15
Planned obsolescence	15
HLS as the potential attack vector	15
Future work	16
Dependability	16
Advanced dependability for image processing and ML	18

Introduction to SoCs and heterogeneous SoCs

See what has been done in ACA and ES.

Offloading computation

We have two ways of offloading computation (not mutually exclusive):

- **Offload execution:** master CPU is on hold while accelerator works
 - Mostly improves performance/energy consumption
- **Parallel execution:** all units are active and working, mostly exploits task-level parallelism
 - Higher power consumption and may introduce synchronization and coherence problems
 - Useful in case of computation to be done on multiple independent data

A hardware accelerator not only needs to accelerate a function, but also handle **transfer of data**. We have different ways to handle it, based on our requirements:

1. **DMA:** very useful in case we have to support a lot of data transfers, since we can do it without CPU intervention
2. **Private local memory:** specialized multi-bank local storage used mainly for buffering and temporary results
3. **Transaction-level abstraction and pipelining**

Balancing communication and computation is crucial for performance optimization. **Optimizing microarchitecture reduces the computation latency**, while **input and output phases** interact with the rest of the system and **can increase latency due to congestion/bus speed**. Thus we need to **reduce the congestion or exploit it** to optimize the execution at the system level.

Communication is the **most critical aspect affecting system performance**: it consumes up to 50% of total on-chip power and the ever increasing number of wires, repeaters, bus components (arbiters, bridges, decoders etc.) increase system cost. Thus, the design flow must include communication design.

Buses are communication lines connecting different parties. The principal **components** involved in communication are:

1. **Master** (Initiator): IP component that initiates a read/write data transfer
2. **Slave** (Target): IP component that only responds to incoming requests
3. **Arbiter:** controls access to the shared bus
 - Implements an arbitration scheme
4. **Decoder:** component that determines which party a transfer is intended for
5. **Bridge:** connects two buses

Bus wires are implemented as **long metal lines** on a silicon wafer and **transmit data using electromagnetic waves**: this puts a **cap on the speed** we can go. As application performance requirements increase, **clock frequencies are also increasing**, meaning that the **time allowed for a signal to travel is also decreasing**. Thus it can take **multiple cycles to send a signal** across a chip. Moreover, **every unit attached to the bus adds parasitic capacitance**, therefore electrical performance degrades. Moreover, since **buses are shared**, the **bandwidth is also shared** and **arbiters add delay** proportional to the number of masters.

Buses are, however, **very cheap** silicon-wise, **simple** to reason about and offer **wide compatibility**, including between very different IPs.

An evolution on top of buses is the **Network-on-Chip** architecture: we **leverage networking principles to improve inter-component intra-chip communication**. This brings a **throughput improvement** w.r.t a standard shared bus. A fundamental component of NoCs are **routers**: special IPs responsible for **routing packets from different components**. NoCs can improve performance and delays, but **require more silicon and wires** and can **introduce delays due to sub-optimal routing**. Moreover **optimizing communication** with an accelerator **may require an ad-hoc placement** of said accelerator.

ASICs

Logic is custom and fixed. Since the logic is fixed and is very **difficult to change after fabrication**, the design must be** thoroughly tested before sending it** to the foundry. This has a very **high up-front cost** (fixed cost or NRE (Not REcurrent) cost) and complexity. However, **after the design is completed**, the **cost remains almost constant** since we can fabricate full batches of them.

ASIC are **not done full-custom anymore**, but **composed of different standard cells which are foundry dependant**

(since they are optimized for a specific capacitance/delay and other physical parameters). Cells are grouped into a foundry-provided library that provides all information for performing accurate simulations and analysis.

FPGA

It is an integrated circuit that can be **configured by the user to emulate any digital circuit** as long as there are enough resources.

FPGAs are **more expensive** than ASICs **per-transistor**, however thanks to their configurability and reconfigurability **scale much better at the start** thanks to the **much lower non recurrent costs**. However, **as the number of units increases**, FPGA cost **eventually catches up** to that of ASICs **and surpasses it**. Thus we can choose between the two based on our market strategy:

- We have a lot of money and plan on selling billions of units? Go ASIC
 - Think Intel, AMD, NVidia
- We plan to sell a small/limited number of units (even one-off chips)? Go FPGA

The **core** is a **regular array of programmable basic logic cells** (Configurable Logic Blocks) that can **implement combinational** as well as **sequential logic**. These blocks are **connected through a programmable interconnect** (switch boxes).

The **programmability is done through a “configuration memory”** (basically a LUT) that stores the configuration data. **Each component is composed of:**

1. Its **lookup table** (implements the boolean function of the component)
2. A **flip flop** used **in case of sequential logic**
3. A **multiplexer choosing between the flip-flop output and the static lookup table value** (user configurable) used for toggling between combinatorial/sequential

We can see that this architecture is **very expensive in terms of power and silicon**: even the simplest of components **requires a full truth table** (for n inputs this is 2^n entries) a **mux and a flip flop**. Moreover, having a **lot of memory means** that the **reliability of the system is reduced** since **bit-flips and other memory corruptions are always catastrophic**.

Switch boxes are basically composed of one **six-pass transistor for interconnection point**, creating a **matrix of wires that are selectively activated** based on configuration. Older switch boxes put on the programmer the responsibility of ensuring that no shorts are present.

The **configuration is called a “bitstream”** and must include the data of all CLBs and SBs, even unused ones. The bitstream is generated by a synthesis tool like Vivado and then flashed onto the board.

Design flows

We ideally would like to specify the functionality using a high level, human readable language and have that compiled all the way down to hardware synthesis. Moreover, as chips acquire more functionalities as said functionalities get more complex, we want to **customize an existing architecture template with other IPs (platform-based design)**.

Application and an architecture template are **refined to meet** the given **constraints** and then are **mapped to HW, interface and software based on the estimation of performance, area and power**. This is **hardware/software co-design**. Thus, this is an act of **balancing**:

1. The **performance** of customized **HW units**
 - Doing things in hardware is faster but costs more
2. **Programmability** of low-cost **SW components**
 - Doing things in software is slower but costs less

Open challenges

1. System-level **optimization**: understand how to **generate and optimize an efficient SoC architecture** also given the technology constraints and the synthesis process

2. **Programmability**: understand the interactions between hardware and software, and how to optimally control the component execution
 - Divided into **2 orthogonal concerns: component generation and system integration**
 - Our components need to have the following **properties**:
 - **Modularity**: possibility of creating an SoC as a collection of components
 - **Flexibility**: possibility of adapting one component to changes in the behavior of the others
 - **Scalability**: possibility of creating larger SoCs without significant performance degradations
 - **Reusability**: possibility of reusing pre-existing components
3. **Reliability/Fault tolerance**: understand how to **protect** the system execution **against random faults**
 - No two transistors are equal, we can have slower or faster ones, due to process variations or degradation (due to aging)
 - Transistors can come out broken
 - Input can also be corrupted/garbage
4. **Testability**: Design needs to be testable to ensure proper behaviour
 - Needs to be **accounted for during design since components** (depending on application) **need to be reliable in all conditions**
 - Needs to be **accounted for during the design phase since it requires additional resources** (e.g. JTAG)
5. **Hardware security**: understand how to protect the IP but also the data within the SoC itself
 - **Increasing complexity demands design & reuse approaches**
 - **Possibility of different attacks at different levels**:
 - Supply-chain attacks
 - Hardware trojan horses (HTH)
 - Microarchitectural side-channel attacks
 - IPs can be stolen/cloned
6. **Input language for system verification**:
 - Verilog/VHDL or HLS?
 - **Synthesis is very dependent on how the code is written**
7. **Coprocessor coupling**: two main models
 - **Tightly-Coupled Accelerator (TCA)** shares key resources with the CPU (registers, MMU, L1 cache etc...)
 - **Loosely-Coupled Accelerator (LCA)** is outside the CPU and uses an integrated DMA controller to transfer data between their memory and the system memory
8. **Memory**: which type and how much memory will we use?
 - Registers: many ports at the cost of more area, good for small to medium structures
 - IP blocks: area efficient provided by technology providers, good for medium to large arrays; limited ports
9. **Data footprint gap**: the data the algorithms process is becoming much more than what is needed for the chip itself to run
10. **Increasing manufacturing costs**: chips are becoming bigger and scaling is expensive
 - Smaller transistors require newer lithography instruments with larger error margins
 - **Reduce costs by reusing pre-designed components** (programmability suffers)
 - **Reduce costs by outsourcing fabrication** (security suffers)

Latency insensitive design

A system with multiple components **works correctly as far as it is running with a clock period that is the maximum of the clock periods of the components** (reg to reg). What happens if the **wire delay is greater than the component delay**?

In a deep **sub-micron** process technology (< 90nm), **process variability is a serious concern**. Technology improvements are on the transistors but not on the wires at the same level. This means that **long wires will play significant role in logic synthesis optimization**. We need a global protocol that is insensitive to delays.

To **design** a complex system in a **correct** way we need to:

- **Relax time constraints** during the early phases
- **Simplify the composition of sequential modules** in pipeline mode
- **Facilitate the insertion of extra pipeline stages between one module and the next one** with the purpose of **buffering** those signals which propagate on long wires

Asynchronous systems require designer to think digital systems completely differently (e.g. removing the concept of global clock and use an event-based architecture). A **latency-insensitive design is a specified synchronous system where components are still synchronous but can tolerate arbitrary communication delays.**

A **simple communication protocol is to stall the transmission if the data link is not ready.**

In a Latency-Insensitive Design (LID), a **design is correct if and only if the sequence of the events (and not their timing) is correct: timing becomes a non-functional metric** to evaluate the quality of the evaluation. LID introduces the following concepts:

- A **relay station** is a module that is inserted wherever it is necessary to tolerate delays
- Each relay station introduces one **stalling event**
- A **modules receiving a stalling event as input emits stalling events as outputs at the next cycle**

We call a **shell a wrapper that encapsulates a module and interfaces with channels**, providing services to it like: getting incoming module, filtering void packets, etc. It **guarantees input synchronization and data propagation.**

There are tools that implement latency-insensitive primitives. The area overhead for these wrappers is usually 3%.

Communication synthesis

Multiprocessors can communicate in **two methods**:

1. **Shared addresses**: offer to the programmer a single memory address space that all processors share/
 - Different processors communicate through shared variables in memory, with all processors capable of accessing all memory locations
 - Simpler, but less scalable
 - Memory access time can be uniform or non-uniform. NUMA machines can scale to a much larger size so there is more potential for performance.
2. **Message passing**: Communicating between multiple processors by explicitly sending and receiving information
 - More complex (requires protocols) but more scalable

Processors can be **connected with**:

- **Busses**:
 - Low area, simple interface
 - Lower scalability (due to bus contention) and cache coherency problems
- **NoC**:
 - Higher performance, much more scalable
 - Higher area of occupation, we still have cache coherency problems

The connection type is orthogonal to the type of communication type.

After system partitioning we got a **set of tasks assigned to system components** (processors executing software + hardware components); these processes are **communicating through abstract channels**. **Communication synthesis has to generate hardware and software which interconnects the system components and enables processes to communicate with each other, with peripheral devices and other interfaces.** Communication synthesis, as a top-down design task, is **performed in three main steps**:

- **Channel binding**

Abstract channels have to be implemented using physical communication components. Resources are allocated, abstract channels are partitioned, grouped and bound to the allocated resources; messages corresponding to channels in one group are multiplexed on a shared communication component.

The **main criteria** used for channel grouping is to **avoid bus conflicts and reduce the total number of connecting wires**. This **can be done by grouping together components** that don't access the channel concurrently, or grouping together channels that are accessed by the same process. Depending on its features, **a communication unit can support a certain number of channels to be multiplexed on it**, without reducing the communication rate below a required minimum.

- **Communication refinement**

After channel binding the **interconnection topology of the system is known and it is determined which channels are bound to a given communication support.**

Communication is still quite **abstract** and has to be **refined with several implementation details** (width of the lines imposed by the foundry/technology, control strategy and communication protocol)

- **Interface generation**

The interfaces needed for a correct functionality of the system can be generated; **both software and hardware components have to be generated** (access routings, controllers, adapters and low-level support for communication related tasks like interrupt control, DMA etc.).

After communication synthesis, the initial system specification results in a specification which can be directly synthesized to a physical implementation.

Communication protocols

Like said before, a bus consists of wires connecting two or more processors or memories. **Bus protocols are usually described using timing diagrams. Others methods can also be used, like HDL descriptions or grammar-based descriptions.**

We have the following two methods for **controlling access to the bus**:

1. **Strobe**: the **master uses one control line**, often called the request line, to **initiate the data transfer**; the **transfer is considered to be complete after some fixed time interval** after the initiation
2. **Handshake**: **master uses a request line to initiate the transfer**; the **slave uses an acknowledge line** to inform the master when the data is ready

A multiprocessor communicates with other devices through some of its pins. To manage it from software, we have different methods called I/O addressing:

- **Port-based I/O** (or parallel I/O): the processor has **one or more N-bit ports** and it **read/writes to ports just like registers**
- **Bus-based I/O**: processor has **address, data and control ports that form a single bus**
 - **Communication protocol needs to be built into the processor**
 - **A single processor instruction carries out a protocol read/write** on the bus
- **Parallel I/O peripheral**: used when the processor supports bus-based I/O but parallel I/O is needed
 - Each port on peripheral is connected to a register that is read/written by the processor
- **Extended parallel I/O**: used when processor supports port-based I/O but more ports needed
 - One or more processor ports interface with parallel I/O peripherals extending number of ports

Communication with a device which produces data asynchronously is done using **two paradigms**:

- **Polling**: repeatedly check whether data is available
- **Interrupt**: the processor checks for the presence of a particular condition
 - When an interrupt happens then the ISR is called (ISR is at a fixed location)
 - Vectored interrupts make it possible to determine the address at which the ISR resides

DMA is implemented via a single-purpose processor which transfers data between the internal memory of the peripheral to the processor memory. Processor needs to **manage bus contention** when using DMA to avoid bottlenecks.

Hardware security

See more or less what has been said in the computer security course and in the “open challenges in hardware design” class.

HLS

High-Level Synthesis creates an **RTL implementation from C, C++, System C, OpenCL kernel code.** **Extracts control and dataflow** from the source code and **implements the design based on defaults and user applied directives.** **Many implementation are possible from the same source description,** enabling easy design-space exploration.

HLS brings the following advantages:

- Productivity:
 - **Easy to model higher level of complexity**
 - **Smaller source** in size compared to RTL
 - Generates RTL faster than the manual method
- Quality of results:
 - **Automatic parallelism extraction**
 - **Multi-cycle functionality**
 - **Loop optimization**
 - **Optimization of memory access**

HLS enables us to **do both functional verification**, by feeding the C code test inputs and checking the output against a reference, **and cosimulation**, by synthesizing the RTL design with the desired constraints/directives and checking again the outputs. We then evaluate the implementation and iterate as necessary.

Control and datapath extraction

Control and datapath can be **extracted from C code for each function**. At some point in the top-level control flow, control is passed to a sub-function. **Sub-functions may be implemented to execute concurrently with the top-level one and/or other sub-functions**.

High-level concepts:

- **Functions**: all code is made up of functions which represent the design hierarchy - the **same hierarchy in hardware**
- **Top Level IO**: the **arguments of the top-level function** determine the hardware RTL interface ports
- **Types**: all variables are of a defined type (even custom). The type can influence the area and performance
- **Loops**: how these are handled can have a major impact on area and performance
- **Arrays**: they can influence the device IO and become performance bottlenecks
- **Operators**: operators in C code may require sharing to control area or specific hardware implementations to meet performance

HLS maps C code to hardware through scheduling and binding processes:

- **Scheduling** determines in **which clock cycle an operation will occur**, taking into account control, dataflow, and directives; the technology and user constraints impact the schedule
- **Binding** determines **which functional unit is used for each operation**, taking into account component delays and the given directives

High-level functionality description

Our problem is the following:

- **Input**:
 - An **intermediate representation**
 - A set of functional **resources**
 - A set of **constraints**
 - * Maximum area, maximum latency/minimum throughput
 - One or more **objectives**
 - * Maximize area, minimize latency/maximum throughput
- **Output**:
 - **Hardware description** of the data-path+controller
 - * Datapath: contains the functional/memory/interconnection resources
 - * Controller: a FSM that determines the next action to run
- **Tasks**:
 - **Place** operations in **time** (scheduling) and **space** (binding)
 - Determine **detailed interconnection and control**

Translation into hardware works analogously to any compilation with a standard compiler based on e.g. LLVM.

Given a functionality, **HLS always generates ports for each of the top-level parameters as follows:**

- Parameters **passed by copy** are converted into **input ports** (connected to registers written by the CPU)
- Parameters **passed by reference** are converted into **memory interfaces** (access to a memory external to the component)

HLS also **adds control ports to manage start/done/reset**.

HLS can also **automatically generate standardized interfaces on top of the basic ones** (AXI-lite for parameters or AXI-Master/Stream for memory accesses).

When dealing with **external memories**, the **scheduling** phase must have **assumptions on the latency of the operations**:

- **Local data** (PLM or scratchpad) have **fixed-latency** access
 - Generates **simple interfaces without sync protocols**
- **Remote data** (cache or off-chip memory) have **variable-latency** access
 - Necessitates **more complex interface** with protocols to exchange data

Scheduling and binding

Scheduling is the **assignment of operations to time** (control steps), possibly **within given limits** on hardware resources and latency. It:

- Uses the **data-dependencies** identified during compilation to **identify parallelism**
- Exploits **mutual exclusion to avoid conflicts** on resources
- **Optimizes loops**

Generally it is **one of the first steps** in the HLS engine.

Binding is the **assignment of operations to hardware resources** (functional units) **such that there are no conflicts in using them** and the **total number is minimized**. To do so, it:

- Uses **scheduling information** to identify sharing opportunities
- Exploits **mutual exclusion** (e.g. operation in different BBs are never executed at the same time and can share resources)
- **Can be defined before scheduling**
 - Imposes constraints on scheduling

Resource sharing is the possibility of using the same functional unit to implement two (or more) operations without any conflicts. Sharing opportunities **can be defined before or after scheduling**:

1. **Before scheduling**:
 - **Pre-defined binding**: two operations that share the same resource **cannot be executed in the same clock cycle and must be serialized**
2. **After scheduling**:
 - **Binding algorithms on scheduled graph**. Two operations that are not executed in the same clock cycle can share the same functional unit

Scheduling

The scheduling steps **takes as input the IR, the clock period and the FU latencies, and it produces the start time of each operations such that all data-dependencies and resource constraints are satisfied**. Its **primary goal** is to **optimize the circuit latency**, the **secondary objective** is to **manage the area/latency trade-off**.

Scheduling determines the timing evolution of the circuit, so **it has a direct impact on the latency** (or throughput) of the implementation. It has also an **indirect effect on area**: operations in the same clock cycle require to be assigned to different physical units so the **maximum number of concurrent operations of the same type is an upper bound on the required number of hardware resources**.

We can **approach** scheduling in 3 different ways:

- **Without constraints**:
 - Assumptions: **infinite resources**
 - Uses: calculate a **lower bound on clock cycles**
- **With resource constraints**:

- Schedules operations (possibly serializing them) such that the overall number of used resources is with a given budget
- Applications: limit the use of resources
- With **timing constraint**:
 - Schedules operations such that the end time of the last ones are within a given time budget
 - Applications: real-time scheduling

Scheduling is an **NP-hard problem**, so we need some **heuristics** to make the problem algorithmically feasible. There are several algorithms, some are:

- As Soon As Possible (**ASAP**) and As Late As Possible (**ALAP**)
- Borrowing from compilers:
 - **List-based scheduling**
 - Force-directed scheduling
 - Path-based scheduling
 - Percolation scheduling
- **Meta-heuristic**: simulated annealing, tabu search etc...
 - As opposed to simple heuristics, meta-heuristics **explore the design space** and may or may not converge to one/the same solution
 - They generally provide **better solution**, but require **much more computational resources** than simple heuristic

ASAP Each operation is **scheduled in the first clock cycle in which is available**, i.e. when all its predecessors have been scheduled and have completed their execution.

- Assumption: **all operations have a bounded delay** (in clock cycles)
- Imposes **no constraint on resources/area**
- **Minimizes latency**: provides a lower bound to latency

Algorithm:

1. Initialize the set of ready vertices with the source node
2. Pick one node from the set of ready vertices and schedule it with the following equation

$$start_time(w) = \max_{p \in Pred} end_time(op_p)$$

3. Define the end time of the current node

$$end_time(op_i) = start_time(op_i) + delay(op_i)$$

4. Add all successors of the current node to the set of ready vertices
5. Repeat from step 2 until the set is empty

ALAP Each operation is **scheduled in last clock cycle where it can be scheduled without causing an extra delay**. It is the **dual problem of ASAP**: it solves a latency-constrained problem where the latency bound is set to latency computed by ASAP algorithm.

- Assumption: **all operations have a bounded delay** (in clock cycles)
- Imposes **no constraint on resources/area**

Algorithm:

1. Initialize the set of ready vertices with the sink node
2. Pick one node from the set of ready vertices and schedule it with the following equation

$$end_time(op_i) = \min_{s \in Succ} start_time(op_s)$$

3. Define the start time of the current node

$$end_time(op_i) = end_time(op_i) - delay(op_i)$$

4. Add all predecessors of the current node to the set of ready vertices
5. Repeat from step 2 until the set is empty

Mobility Mobility is a **metric associated with each operation and is defined as the difference between its ALAP and ASAP schedules.**

- **Zero mobility** implies that an **operation can start only at a given time step** without introducing any delay on the overall schedule
- Mobility **greater than zero measures the slack** on the start time

List-based scheduling Most common heuristic for constraint-based scheduling, it is a simple **greedy algorithm** (does **not guarantee optimality** but it is **linear** in complexity). It can be used both for **minimizing latency given constraints on area/resources** (ML-RCS) or **minimizing resources given a bound on latency** (MR-LCS).

Algorithm:

1. Construct a priority list based on some metrics (operation mobility, number of successors, etc...)
2. While not all operations scheduled
 1. For each available resource, select an operation in the ready list following the descending priority.
 2. Assign the operation to the current clock cycle
 3. Update the ready list
 4. Continue until there are no more ready operations or available resources
 5. Increment the clock cycle

Static vs dynamic mobility An operation with **high mobility** (and static mobility) is **generally postponed** to the next clock cycle. This incurs a **risk of starvation** possible solution is to **update the mobility after each iteration** (dynamic mobility): we decrease the mobility each time we postpone it.

Challenges All algorithms assumed functional units that complete in one (single-cycle) or more cycles (multi-cycle) and that execute at most one operation. However, we can have **FUs** that can execute more than one operation (**multi-function**) or start another operation before the previous one is completed (**pipelined**).

If two operations are serial and the total execution time is less than the clock period, they can be executed one after the other (**chaining**).

Operations may have unbounded latency, e.g. accesses to external memory, and may require **synchronization protocols**.

Binding

It is defined as the **spatial mapping between operations and resources**. It tries to search for sharing opportunities.

It can be **constrained**:

- Resource-dominated circuits
- Fixed number and type of available resources

This is again an **NP-complete** problem, meaning we need to use **heuristics**.

We take as **input the scheduled graph and well defined concurrency information** for operations. We **consider operation types independently**. We **perform analysis on operation pairs**:

- **Compatibility**: same type, non-concurrent, etc.
- **Conflict**: concurrent, different types, etc.

These two are **dual** problems.

Compatibility Let us define the **compatibility graph** G_+ :

- Vertices represent operations
- Edges represent compatible operation pairs
 - Two operations are compatible if they are not concurrent and can be implemented by resources of the same type

The binding problem can be formulated as a **partitioning of the compatibility graph**:

Each partition is a clique (fully connected subgraph) of operations that are all compatible with each other. So, they can share the same resource

The **clique covering** is thus a partitioning of graph G_+ into the minimum number of cliques, where each clique represents a functional unit.

It is possible to **solve the partitioning** imposing a **minimum number of cliques** (more units, less interconnections), **or to assign weights** to edges to prioritize some connections.

Conflict We can define the **conflict graph** G_- , which is the **complementary of the compatibility graph**, as such:

- Vertices represent operations
- Edges represent operations pairs in conflict
 - Two operations are in conflict if they are not compatible

The binding problem can be **formulated as a coloring problem** of the conflict graph.

Each node will be assigned to a color and two adjacent nodes cannot have the same color

The **color** will represent the **identifier of the functional unit**.

The goal is to **minimize the overall number of colors** (each node with a different color is an admissible but trivial solution).

Microarchitecture creation

After defining the operation scheduling and binding, HLS **proceeds with the generation of the RTL microarchitecture** of the following elements:

- **Controller**: a **FSM** where
 - The states are a collection of operations to be executed in the given clock cycle
 - The transitions define the evolution over time of the behaviour; can be either:
 - * Inside a basic block, in that case it is a sequential list of states to be executed from the beginning to the end
 - * Among basic blocks, in that case it transitions between the last operation of one basic block to the first operation of the next one
- **Datapath**: a collection of **hardware resources for computation and storage**, mainly composed of:
 - Functional units
 - Registers
 - Wires and multiplexers to interconnect all resources
- **Interfaces** with local and/or external memories

Memories and memory operations

C allows us to easily specify, design, and optimize **accelerators for irregular applications** thanks to the massive use of **pointer-based operations** (arithmetic, dynamic resolutions, accesses to external memory etc...).

SystemC allows us to easily specify, design, and optimize **data-intensive accelerators** thanks to the **massive use of arrays with predefined size** and **the use of DMA transfers** with main memory to exchange data blocks.

Pointers In software, a C program targets a virtual architecture consisting of a single (unified) memory in which all data are stored. Pointers are the addresses of elements in memory. Even though register declarations may allow programmers

to specify the variables to be placed in registers, the assignment of variables to registers is generally done by the compiler. Moreover, the notions of caches and memory pages are transparent to programmers.

In hardware, **designers want to have control on where data are stored and optimize data locality**. Typically, a chip design contains multiple memory banks, register files, registers, and wires. **To efficiently map C code onto hardware, the unified storage space must be partitioned**. During synthesis, **each partition is then mapped to a register, a wire, or a memory**.

Pointers may be used to reference any variable no matter where its information is available (references to memory elements, registers, wires, or ports).

The **hardware synthesis of pointers** includes the following steps:

- **Partitioning** of the memory into locations (or partitions)
- **Mapping** of the partitions onto hardware resources
 - To a variable (wire or register)
 - To an array (akin to memory or register file)
- **Generation of the proper hardware logic to access the data**

Virtual addresses must be translated into operations to the proper hardware resources.

A possible architecture is having an **internal memory bus** where the **pointer is dynamically resolved**. This creates a **daisy-chain architecture** with possibility to access the external memory.

Private local memories PLM are commonly implemented as **specialized multi-bank memories for storing part of the data**. The **memory design is transparent to the accelerator logic** (a PLM controller manages it). **Each PLM unit can have alternative implementations by using block/cyclic partitioning**.

To ensure optimal performance, we must **ensure that arrays are transformed in such a way to create independent data structures that can be accessed in parallel**:

- Each new substructure is managed as a new array
- How to distribute the data contained into the original array to guarantee that parallel operations operate on distinct arrays?

It is necessary to **determine the access pattern**, i.e., the distribution of the memory operations on the array over time. **When the access pattern is irregular or unknown, we need to duplicate the data**.

HLS is responsible for the creation of accelerator logic. **PLM generation is done in a separate process** that considers **memory-related parameters, defined data structures and information about access patterns**. This process **creates specialized PLMs** that include various **technology related optimizations and system-level optimizations**.

Generally, we can use one PLM unit (eventually composed of many banks) for each data structure. Ideally, **we want reuse as much as possible the same memory IPs**. We say that **two data structures are compatible if they can be allocated to the same PLM unit**. A common case is accelerators never executed at the same time.

The accelerator(s) **memory subsystem is defined during SoC integration** since it allows for **more optimization** possibilities (like inter-accelerator one).

We can specify **two types of compatibility**:

1. **Address compatibility**: the data structures are compatible and can use the same memory IPs
 - Different partitioning schemes might be used depending on the data structure
2. **Memory-interface compatibility**: the ports are never accessed at the same time and the data structures can stay in the same memory IP

We can define the **Memory Compatibility Graph (MCG)**, a graph that represents the possibilities for optimizing data structures. Each **node** represents a **data structure to be allocated**, annotated with its data footprint; each **edge** represents the **compatibility between the structures** (address or memory-interface). A **clique** of this graph represents a **set of data structures that can share the same memory IPs**.

To **generate the memory subsystem** we follow this process:

1. **Determine how to partition the MCG**:
 - **Clique enumeration**: define the list of admissible cliques

- **Clique characterization:** for each clique, determine the architecture and memory cost
 - **Memory cost minimization:** determine the partitions that minimize the total memory cost
2. **PLM controller generation:** a lightweight PLM controller is created for each compatibility set

Dynamically controlling the banks A scenario is a **given configuration of the accelerator to execute a specific problem instance**. PLM units must be **sized for the worst-case scenario**, but they **may not be entirely used in all scenarios, allowing for fine-grained power-savings**.

Each accelerator can **turn off the banks that are not entirely used** in the current scenario. **Design-time partitioning** of the banks is done to **maximize the ones that are power gated**. **System conditions can also alter the execution dynamics**, thus we can also **dynamically control the power gating** based on the execution phases with **3 operating modes: active, idle and deep-sleep**.

Each PLM unit can be extended with power-control logic. Each SRAM bank is controlled in a fine-grained way through its output pins (PGL and PGM). **We have 2 controller working in tandem to ensure optimal behaviour:**

- **Scenario Memory Controller (SMC):** identifies the current execution scenario by analyzing the configuration registers (provided by the user with memory- mapped operations)
 - In each scenario, only the used banks are kept active, while the others are power gated
- **Operating Mode Controller (OMC):** manages the SRAM operating modes; an FSM handles the transitions

This is called the **DarkMem** architecture.

HLS can **automatically generate DarkMem units** as an extension of the “PLM Customization” phase. Additional information about the execution scenario can be estimated by the designer.

Determining the optimal number and type of banks for each PLM unit can be formulated as an Integer Linear Programming problem based on:

- List of scenarios and frequency of execution
- Data to be stored (bitwidth and number of words) in each scenario
- List of available memory IPs and corresponding active/gated static power configurations

Off-chip memory Application data footprint grows more than available on-chip storage. Cache-based memory hierarchies and virtual memory have been able to deal with this for CPUs. **Accelerators, on the other hand, need fast memory access to fetch long data bursts and cannot work well with caches since:**

1. **PLMs are too large** to maintain an inclusive directory and avoid recalls
2. **Long data transfers exploit little locality** and frequently incur eviction penalty
3. **Address translation with typical page sizes requires extremely large page tables** (equally-sized pages) **or slow translation logic** (scatterlists) with CPU control

To remedy this we:

- **Allocate equally sized large chunks and create a small page table**
 - **Similar to huge pages**, but with configurable page-size and not allocate at boot
 - Similar to a scatterlist, but the chunks’ length doesn’t need to be stored and page lookup only requires bit-selection logic
 - **Built-in load balancing across DRAM controllers**
- **Dedicated DMA controller with TLB per tile**
 - Allows to fetch the entire page table with one DMA transaction
 - It hides look-up latency during accelerator computation
 - Breaks long bursts into equally sized DMA transactions to balance link access across accelerators

Balancing multiple accelerators

Balancing communication and computation is crucial for performance optimization. Optimizing microarchitecture reduces the computation latency (through a combination of HLS transformations and PLM customization). **The input and output phases interact with the rest of the system, since backpressure due to congestion may increase latency. We can either reduce it, or exploit it to optimize the execution at the system level.**

Hardware security

Increasing system complexity demands design & reuse approaches. **IP components are coming from many vendors and assembled to create the SoC.** Hardware security is the **next big issue for hardware design** since the **supply chain is getting more and more distributed** to reduce costs, leading to more security threats.

Some **common hardware threats** are:

- **Reverse engineering and IP theft**
- **Hardware trojans:** malicious modifications of an existing chip design to introduce an additional functionality
- **Data injection:** injection of spurious data to exploit software or hardware/software vulnerabilities
- **Side-channel:** methods to create additional communication channels to steal sensitive data

HLS is used to automatically generate RTL designs starting from a high-level specification and thus **can be used to integrate the automatic generation of security features** like:

- Fine-grained dynamic information flow tracking (**DIFT**)
- **Algorithm obfuscation**
- **IP watermarking**

This also becomes a problem since the **HLS generation becomes itself a possible attack vector**.

Dynamic Information Flow Tracking

Dynamic Information Flow Tracking (DIFT) **allows marking data coming from untrusted sources with tags, such that the hardware can trap to the OS if tainted data is used in critical operations.** DIFT can protect from several software-based attacks.

Applications interleaves tasks in both hardware and software. **Since our accelerator may do its work between when the malicious data is injected and when the exploit is enabled, we need to explicitly support fine-grained DIFT in our accelerators. Otherwise,** if we are acting like black-boxes, we can act in **two ways**:

- **Optimistic:** we ensure that the **data** that comes out of our accelerator is **always safe**
 - This means that **possibly bad data will always become untainted**
 - We need to guarantee that our logic does not have any bugs
- **Pessimistic:** we **do not guarantee any safety** of the output of the accelerator
 - This means that **potentially good data will always become tainted**
 - If all output data is always tainted, **the system will always halt**

To implement fine-grained DIFT we need to:

- **Extend the data-path** with **shadow logic** for handling the **propagation of the taint flag**
- A **memory architecture** with **taint memories** to store the taint flags of variables

This incurs **almost no performance overhead** with **available optimizations to limit the area overhead**. There are **HLS-based methodologies for automatic generation** of all the necessary logic.

DIFT is **based** on the notion of **data-flow consistency**, which is the idea of **doing the same operations we do to the elements of the datapath to those of the taint, for example**:

- Writing to a register causes an update to the taint of the register
- Writing to a certain memory address causes an update to the taint associated with that address in the taint memory
- Paths between the various units of the datapath are the same as those between different units of the DIFT logic

The **area overhead** for DIFT is **at most around 30%** with bit-level granularity. With less granularity we have less overhead.

Obfuscation

To **avoid IP theft**, one simple way is to **send to foundries/integrators an obfuscated netlist**. The **chip will work correctly only if the corresponding key is provided**. This method is **weak if the third party can somehow find out some details about our chip**, allowing them to **rule-out bad keys and narrow down the search**.

Another way is to **obfuscate at the algorithm-level, such that the obfuscation is semantically meaningful** and allow us to better hide the semantics of the chip. An algorithm is characterized by different elements that can be protected, each protected by a different technique of algorithm-level obfuscation:

1. **Constant obfuscation: m-bit constants are extracted and encoded using m working key bits**
 - Obfuscates: data coefficients, signal extensions and mask values
 - Doesn't obfuscate: reset values, signal polarity
 - Less information is made available to the attacker, but we also **lose some potential logic optimizations**
2. **Control-flow obfuscation: we mask control conditions with a XOR with the key bit**
 - Needs to focus on branches during data computation
 - **Easy to apply** (a XOR gate has minimal overhead)
 - **Almost impossible to replicate at gate-level** (branch info is already embedded)
3. **Operation obfuscation: the key controls a mux that chooses between correct/incorrect operations based on the key bit**
 - **Easy to apply with reasonable area overhead** due to additional fake operations plus multiplexer
 - **Each operation type has a pre-defined set of alternatives** to choose from
 - Based on statistical frequency of the various operations
 - **How to select** operation variants is still an **open issue**
4. **Dependence obfuscation: K key bits are used to select among 2^K DFG variants**
 - Basically the key **controls multiplexers that create correct/fake paths**, creating several semantically-different but feasible code variants
 - **It may affect component latency**
 - **Significant area overhead** due to many additional operators, connections and multiplexers
 - It can be **applied only during component generation**

Obfuscation results are validated by measuring “output corruptibility”: it is defined as the **Hamming distance** between **output values generated with correct and incorrect keys**. Some open question are still:

- **How to select which parts to protect** to minimize the overhead?
- **How to “measure”** the level of obfuscation/security?

Watermarking

IP watermarking is the **process of embedding hidden marks inside the chip's logic such that copied chips are easily identifiable**. The structure of a **Hardware Trojans** can be used for it as:

- It is **stealthy during normal operation**
- Can be **activated on a rare condition** (the trigger)
- The **payload is a small and powerful function** intertwined with the functionality

Only one function contains the watermarks but the others must propagate the values.

To generate the watermark, we **analyze the input call graph**. Our **target** are **functions with the largest number of operations or the most frequently executed ones**. Then we **modify/remove operations in such a way that we have a low probability of accidentally finding the watermark** (low probability of coincidence). Since the **HLS tools knows the modified operations**, it is **able to generate golden values in order to validate a watermarked design**.

Unlike locking, which actively prevents copying, **watermarking can only detect if a design has been copied but cannot prevent the copy**.

Planned obsolescence

A problem present with the whole hardware design flow, not only HLS, is that it is **very difficult to check the non-functional properties** of a hardware/software system. This fact can be **abused by manufacturers to sneakily degrade IPs after a certain amount of time, pushing consumers to change devices**.

HLS as the potential attack vector

CAD tools are software, and all software is susceptible to bugs or other security threats (black-hat HLS is possible!). This means that **we cannot fully trust third party CAD tools with our designs**.

Attacks are usually **implemented as malicious passes inserted in the HLS design flow** that alters the design in undesired ways. Some possible attacks are:

1. **Degradation attack:** aims at **degrading the performance of the IP core** after a pre-defined amount of time
 - Usually done by **adding bubble states in specific points of the FSM**
 - Can degrade performance of up to 60% with just 16 extra states added to critical FSMs
2. **Accelerated battery discharging:** since HLS knows which functional units are used in each clock/cycles, we can use this information to **artificially increase the power consumption by turning on unused units or adding extra logic**
3. **Key recovery with reduced-rounds:** many **cryptographic algorithms** execute multiple “rounds”, we can **stealthily reduce the number of rounds to ease key recovery**
 - Requires **collusion between HLS developer and IP developer**

Future work

Ideally, we want a **clear metric** to quantify how much a system is secure and a **push-button solution to create a complete and secure architecture** since hardware security is critical as it isn't possible to “patch” hardware as we do with software.

The reality, however, is very different. **Security certification is impossible** since we can prevent only attacks that we know, meaning that **it is all a cat-and-mouse game between designers and attackers**. We can **never completely prevent attacks**, thus the goal is then to **make the life of attackers extremely difficult**, but at which cost and at which level?

Security must be addressed at ALL levels with:

- Provably-secure algorithms
- Robust OSs and protected communication
- Secure components, architectures, integrations etc...

Separation of concerns is required for scalable solutions, so a complete and integrated solution is missing. Being aware of the problems is, however, as important as proposing countermeasures.

Dependability

Dependability is the **ability of a system to perform its functionality while exposing:**

- **Reliability:** continuity of the service
- **Availability:** to be working properly when the service is required
- **Maintainability:** capability of a system to be easily repaired/maintained
- **Safety:** absence of catastrophic failures
- **Security:** ability to guarantee confidentiality and integrity of data

Dependability is important because **we are not only trying to design a system that works correctly** (i.e. it meets functional requirements), **but one that also works acceptably** (e.g. doesn't break after 2 hours) and doesn't fail since a failure:

- May affect a large number of people
- May incur high costs if it impacts economic losses or physical damage
- May damage the reputation of the manufacturer (no one will buy a system that constantly breaks)
- May cause information loss with high recovery costs

As such, **every system needs to expose dependability**. Moreover, **depending on the industry, standards may require it** (ISO 26262 for automotive and others).

Dependability **needs to be addressed:**

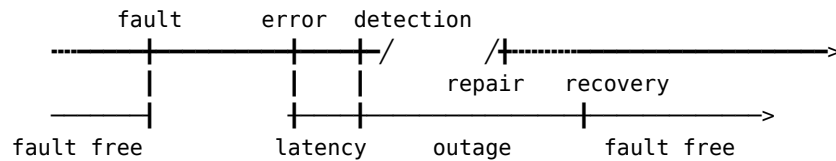
1. **At every step of the design process**
 - By analyzing the system under design, measuring the appropriate dependability properties and adapting the design to optimize it
2. **At runtime**
 - By **detecting malfunctions, tolerating them and reacting** appropriately

Let us define three important terms:

1. **Fault:** and event within the system

2. **Error**: a deviation from the required operation of the (sub)system
3. **Failure**: the system fails to perform its required function

The basic timeline of events goes as follows:



Note that even if $\text{error} \implies \text{fault}$, the reverse is not necessarily true: this means that **the fault-error-failure chain not always goes to the end** as faults could not cause errors or errors may be corrected and not cause failures.

We have **two paradigms** for **dealing with failures**:

- **Failure avoidance**: it tries to avoid failures at design time as much as possible by:
 - Using a more **conservative design**
 - Formally **validating** the design
 - **Testing** in a very detailed manner both HW and SW
 - **Screening for “infant mortality”** of the components
 - * There is a time after the production but before the deployment where bad components are more likely to be found (e.g. by stress-testing) that will make your system unreliable at launch. By screening these components, we can lower our unreliability to acceptable levels, until inevitable wear degrades reliability again. At that time we should recall the system.
 - Implementing **error avoidance**
- **Failure tolerance**: tolerate failures when they happen at runtime by:
 - **Error detection/error masking** mechanisms
 - On-line **monitoring**
 - **Diagnostics**
 - **Self-recovery or self-repair** functionalities

To ensure dependability we can work at **different levels**:

- **Technological/component level**:
 - Design and manufacturing is done employing reliable and robust components
- **Hardening of the functional units, register files or memories**:
 - Redundancy, error detection/correction etc..
- **Architectural level**:
 - Integrate normal components in an architecture that allows to manage the occurrence of failures
 - Some **example architectures**:
 - * **Lock-step dual processor**: two processors execute the same code in a strictly synchronized manner, bus and memories are protected with codes and the interrupt controller is specifically designed with this fault tolerance in mind; if one of the two processors produces a different result, the fault is detected and no output is produced, no tolerance if guaranteed
 - * **Loosely-coupled dual processor**: the two processors run independently and at runtime may be configured to run in lock-step or not
 - * **Watchdog/checker processor**: observes the behaviour of the processor and performs a high-level anomaly detection like: if execution statistics differ from the profiled ones, if the data values or ranges are the expected ones, timeout expirations
 - * **TMR**: lock-step solution with 3 processors, allowing detection and tolerance of single faults
 - * **Dual lock-step**: two lock-step processors (for a total of 4) providing both fault detection and tolerance
- **Application level**:
 - Process replication/diversification
 - Process re-execution
 - Checkpointing
 - Instruction-level hardening
 - Codes (e.g. CRC)

All these approaches have a **clear trade-off**: they **cost more and have reduced performance**. We have to pay for depend-

ability. This means that we need to **find the best compromise** between dependability and cost **by considering**:

- The **application field** of the system
 - If there is a specific design standard?
 - Which degree of dependability is actually required?
 - What is the cost (monetary, lives etc) of a failure?
- The **working scenario**
 - Are there special sources of faults (radiation, pressure, temperature etc)?
 - Which are the nominal working condition
 - Are there other components connected to the system that may induce faults?
- **Employed technologies**
 - Are CPU/memory/interfaces free from the sources of failure, tolerant to them?
 - Which are the components most susceptible to failures?
- **Algorithms and applications**
 - Are the inputs free of inexactness or are they noisy?
 - Is the algorithms tolerant to faults?
 - Can the application tolerate downtime?

Advanced dependability for image processing and ML

Didn't have time to take notes soz.